



Universidad Autónoma de Nayarit

Unidad Académica De Economía

Sistemas Computacionales

**“Semana 11 — Búsqueda Lineal y Binaria • HITO
2”**

Jorge Armando Navarrete Ruiz

Docente.- DR. Eligardo Cruz Sanchez

U. A. ESTRUCTURA DE BASES DE DATOS

Fase 1: COMPRENDE	3
Checkpoint COMPRENDE — 4 preguntas metacognitivas	3
Reto IA — Comprende: El invariante bajo la lupa	4
Fase 2: APLICA	4
Reto 5 — Búsqueda lineal + binaria + benchmark	4
Reto 6 — Búsqueda binaria con variantes: primera y última ocurrencia	4
AVANZADO	4
Usamos para recordar la mejor coincidencia y, al encontrar , movemos para buscar la primera ocurrencia. Sigue siendo $O(\log n)$ con duplicados porque el intervalo se reduce a la mitad en cada iteración. Al final, es correcto porque siempre guarda una ocurrencia válida y no quedan candidatos más a la izquierda.	4
Fase 3: REFLEXIONA	4
Cualquier integrante debe poder responder:	4
Reto IA — Reflexiona: Simulacro de defensa oral con foco en invariantes	6
Fase 4: VALIDA	6
Reto IA — Valida: Casos adversariales para búsqueda binaria y el pipeline	6

Fase 1: COMPRENDE

Checkpoint COMPRENDE — 4 preguntas metacognitivas

¿Por qué mover $izq = mid + 1$ (y no $izq = mid$) cuando $A[mid] < objetivo$? Muestra con la traza qué pasa si usas $izq = mid$ con $A = [3, 5]$ buscando 7.

El invariante es que el rango $[izq, der]$ siempre contiene los posibles candidatos.

Si $A[mid] < objetivo$, entonces **mid ya no puede ser la respuesta**, porque todo lo que está a la izquierda (incluido mid) es menor que el objetivo.

Por eso se descarta mid y se mueve el límite izquierdo a $mid + 1$.

Para $n = 1\,000\,000$: ¿cuántas comparaciones hace búsqueda binaria en el peor caso? Calcúlenlo con $\lfloor \log_2(1\,000\,000) \rfloor + 1$ sin calculadora — estimen el logaritmo.

Sabemos que $2^{10} = 1024 \approx 10^3$.

Entonces $2^{20} \approx 10^6$.

Por lo tanto, $\log_2(1,000,000) \approx 20$.

$\lfloor 20 \rfloor + 1 = 21$

En el peor caso, la búsqueda binaria hace **21 comparaciones** para un millón de elementos.

Si el arreglo tiene duplicados — por ejemplo, $[5, 7, 7, 7, 9]$ — y buscan el valor 7, ¿qué posición retorna la búsqueda binaria? ¿Es siempre la misma? ¿Por qué depende del arreglo concreto?

La búsqueda binaria estándar retorna **una posición cualquiera donde el valor coincide**, dependiendo de cómo se calcule mid y cómo se actualicen los límites.

- Si el algoritmo busca "igualdad exacta", puede devolver el índice 1, 2 o 3.
- Si se modifica para buscar el **primer** o **último** 7, se ajustan los límites:
 - Para el primero: cuando $A[mid] == objetivo$, se mueve $der = mid - 1$.

- Para el último: se mueve $izq = mid + 1$.

¿Qué pasaría si aplican búsqueda binaria sobre el arreglo del catálogo de StreamMX *antes* de ordenarlo? Diseñen un contraejemplo concreto con 5 elementos.

La búsqueda binaria **solo funciona si el arreglo está ordenado**. Si aplicas el algoritmo sobre el catálogo de StreamMX sin ordenar, puede fallar incluso con pocos elementos.

Reto 1A — Comprende: El invariante bajo la lupa

Descubrimos que el invariante correcto es que, si x existe, su posible ubicación siempre permanece dentro de $[low, high]$, evitando descartar su índice.

La corrección depende de la inicialización, el mantenimiento al descartar mitades y la terminación por reducción del intervalo.

La condición que lo garantiza es que el arreglo sea no-decrecientemente ordenado, lo que valida cada descarte.

Fase 2: APLICA

Reto 5 — Búsqueda lineal + binaria + benchmark

Pedimos ayuda para justificar con el invariante el , y las actualizaciones ; lo demás (lineal, esqueleto, pruebas/benchmark y duplicados con expansión)

Reto 6 — Búsqueda binaria con variantes: primera y última ocurrencia AVANZADO

Usamos para recordar la mejor coincidencia y, al encontrar , movemos para buscar la primera ocurrencia. Sigue siendo $O(\log n)$ con duplicados porque el intervalo se reduce a la mitad en cada iteración. Al final, es correcto porque siempre guarda una ocurrencia válida y no quedan candidatos más a la izquierda.

Fase 3: REFLEXIONA

Cualquier integrante debe poder responder:

D1 — Traza de búsqueda binaria: Traza la búsqueda binaria sobre $A = [2, 4, 6, 8, 10, 12, 14]$ buscando el valor 10. ¿Cuántas iteraciones? ¿Cuáles son los valores de *izq*, *der* y *mid* en cada paso?

$A = [2, 4, 6, 8, 10, 12, 14]$, objetivo = 10

Iteraciones: 3

$izq=0$, $der=6$, $mid=3 \rightarrow 8 \rightarrow izq=4$

$izq=4$, $der=6$, $mid=5 \rightarrow 12 \rightarrow der=4$

$izq=4$, $der=4$, $mid=4 \rightarrow 10 \rightarrow$ encontrado

D2 — Invariante bajo presión: Explica por qué $izq = mid + 1$ (y no $izq = mid$) preserva el invariante. ¿Qué pasaría con la terminación del bucle si usaras $izq = mid$ en $A = [5, 7]$ buscando 9?

$izq = mid + 1$ preserva el invariante porque descarta solo valores que ya se probó que no contienen al objetivo.

Con $izq = mid$ en $[5, 7]$ buscando 9:

el intervalo no se reduce

el algoritmo puede entrar en bucle infinito

D3 — Precondición y consecuencias: El catálogo de StreamMX no se ordena antes de la búsqueda. Diseña un arreglo de 6 elementos donde búsqueda binaria retorna "no encontrado" aunque el elemento está presente. ¿Por qué ocurre?

Ejemplo: $[3, 1, 2, 4, 6, 5]$, buscar 5

Puede descartarse el lado correcto porque el orden no es monotónico

Resultado: "no encontrado" aunque sí existe

Causa: se rompe el invariante de partición válido solo en arreglos ordenados

D4 — HITO 2 completo: Explica el pipeline del equipo: ¿qué algoritmo ordenó el catálogo?, ¿por qué lo eligieron según el Veredicto de S10?, ¿qué retorna `buscar_por_puntaje(catalogo, 7.5)` y cuántas comparaciones hacen en el peor caso?

Ordenamiento: Merge Sort ($O(n \log n)$, estable, rendimiento garantizado)

`buscar_por_puntaje(7.5)` $\rightarrow$ 3 resultados

Búsqueda binaria en peor caso: ≈ 20 comparaciones ($n=10^6$)

D5 — Comparativa empírica: Con los datos del benchmark del equipo: para $n = 1\,000\,000$ buscando el último elemento, ¿cuántas veces fue más rápida la búsqueda binaria que la lineal? ¿Ese factor se acerca al teórico $n/\log_2(n)$?

$n = 1,000,000$:
lineal $\approx 1,000,000$ pasos
binaria ≈ 20 pasos
Factor de mejora: $\approx 50,000\times$ más rápida
Coincide con teoría: $n / \log_2(n) \approx 50,000$

Reto IA — Reflexiona: Simulacro de defensa oral con foco en invariantes

Lo más difícil fue D2 (), porque requiere probar progreso/terminación. Debemos repasar invariantes y variante de terminación () y la justificación del pipeline (precondiciones, garantías y complejidad: ordenar+buscar+duplicados). $izq = mid - 1$

Fase 4: VALIDA

Reto IA — Valida: Casos adversariales para búsqueda binaria y el pipeline

No hallamos errores: ejecución y predicciones coincidieron, sin violaciones de invariantes ni bucles infinitos. Los casos cubrieron bordes (n par, extremos, $n=2$) y confirmaron y terminación. El pipeline completo también pasó todos los casos sin discrepancias. $mid \pm 1$